

Department of Electronical Engineering, Telecommunications and
Computers

Database Documentation

50565: Ângelo Filipe Maia Azevedo (a50565@alunos.isel.pt)
50539: António Miguel Alves (a50539@alunos.isel.pt)

Database Documentation for the Remote Lab Project

Advisor: Prof. Pedro Miguens Matutino

June 2025

Contents

List of Figures	1
0.1 Introduction	2
0.2 Database Overview	2
0.3 Entity-Relationship Model	2
0.4 Entities and Attributes	2
0.4.1 User	3
0.4.2 Token	3
0.4.3 Group	3
0.4.4 Laboratory	4
0.4.5 Hardware	4
0.5 Associations	5

List of Figures

1	ER Model	2
---	--------------------	---

0.1 Introduction

This document provides a comprehensive overview of the database structure, including its entities, attributes, and relationships. It also details key implementation decisions.

0.2 Database Overview

The database has been designed using an Entity-Relationship (ER) approach. This methodology enables a clear understanding of how different entities interact within the system. The following section presents the ER model diagram.

The database is implemented using PostgreSQL and has been tested with sample data in a Docker container.

The next section presents the ER model in detail.

0.3 Entity-Relationship Model

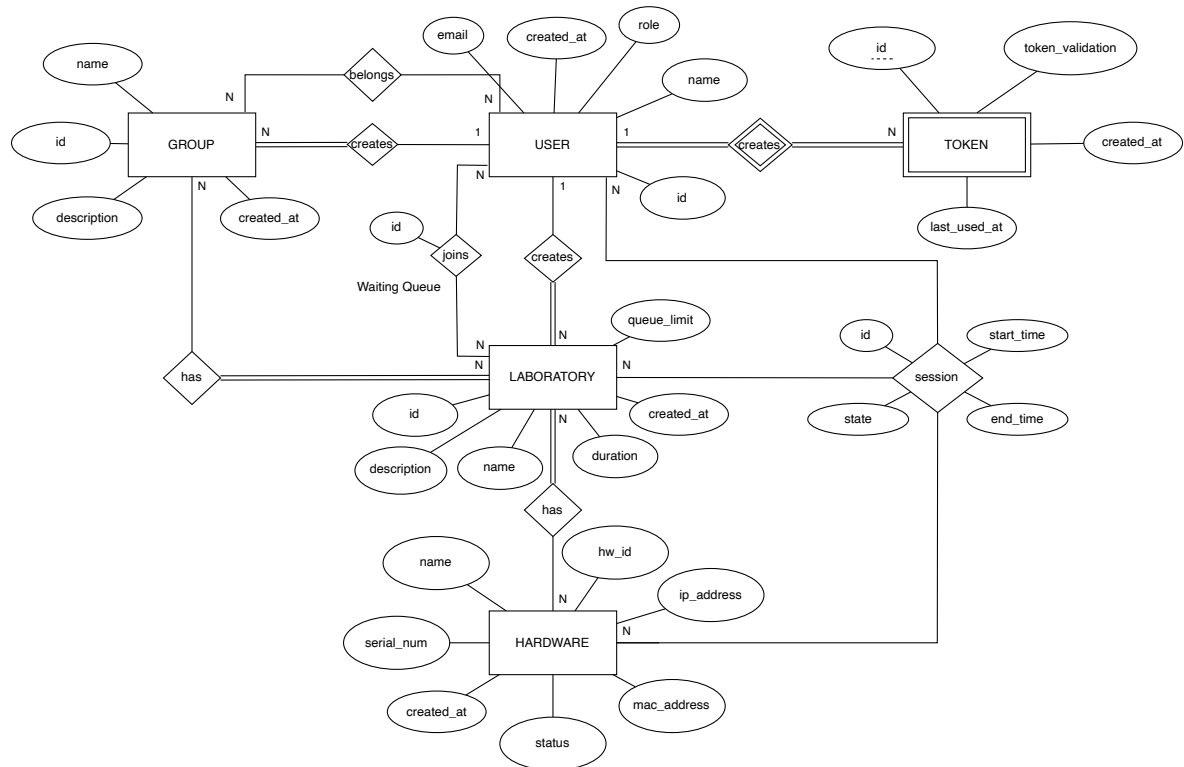


Figure 1: ER Model

0.4 Entities and Attributes

This section provides a comprehensive description of each entity and its attributes.

0.4.1 User

The **User** entity is fundamental to the database and represents system users.

It has an **id** as its primary key, implemented as an identity column. An identity column automatically generates values from an implicit sequence. Therefore, whenever a new user is created, a unique id is automatically generated. The id is an integer data type, chosen to simplify database queries by using a simple numeric identifier instead of email or username.

The entity includes character sequence attributes: **name** and **email**. All these fields are required (not null), and the email must be unique. Initially, the name is automatically set to match the user's name returned by OAuth login. While names are used for display purposes and may not be unique, emails serve as unique identifiers for specific tasks since they are unique. The specific constraints for these attributes are defined in the application domain.

As a discriminator attribute, this entity contains a **role**. This role determines the user's role within the system. This attribute is a char, and the possible options are *S* (Student), *P* (Professor), *A* (Administrator).

Additionally, it includes a **created_at** field as a required timestamp.

0.4.2 Token

Token is implemented as a weak entity since it cannot exist independently and requires a *user_id* for identification. Each token is associated with a user and is essential for authentication purposes. Users must have a valid token when interacting with the system.

As a weak entity, it requires a partial key, **id**, generated automatically. It also contains a **token_validation** field. This attribute stores a randomly generated hash created by the system and is used to verify token validity. For instance, when a user's token is included in an authorization header, the system compares it with this stored value during each interaction. It is implemented as a required character sequence.

The entity also includes **created_at** and **last_used_at** timestamps, both required fields. These attributes enable token validity checking based on timing rules defined by the application domain.

0.4.3 Group

The **Group** entity represents various types of user groupings, such as student classes, work groups, or professor groups.

Each group has an **id** as its primary key, implemented as an auto-generated integer identity column. This unique identifier facilitates efficient querying and group identification.

The **name** is a required character sequence that stores the user-defined name for the group. Group names can be modified after creation, with specific length and format restrictions defined by the application domain.

Groups can include an optional **description** stored as a text type. This field allows users to provide detailed information about the group’s purpose or characteristics. While technically unlimited in length, practical limitations may be imposed by the application domain.

The entity also includes a **created_at** timestamp, which can be used for analytical and administrative purposes.

0.4.4 Laboratory

The **Laboratory** entity represents physical or virtual laboratory spaces within the system.

Each laboratory has an **id** as its primary key, implemented as an auto-generated integer identity column. This unique identifier simplifies laboratory identification and database queries.

The **name** attribute is a required character sequence that stores the laboratory’s designation. Specific naming conventions and restrictions are defined by the application domain.

The entity includes a **duration** attribute, which specifies the standard duration of laboratory sessions in minutes. This integer field is required and helps manage session scheduling.

Additionally, it includes a **queue_limit** to control the number of users in the waiting queue and a **created_at** timestamp for tracking laboratory creation and administrative purposes.

0.4.5 Hardware

The **Hardware** entity represents physical equipment within the laboratory system, such as computers or FPGAs.

Each piece of hardware is identified by an **id** primary key, implemented as an auto-generated integer identity column to facilitate equipment tracking and queries.

The entity includes a required **name** attribute as a character sequence for equipment identification.

A required **serial_num** character sequence stores the hardware’s serial number for inventory management.

The **ip_address** and **mac_address** are character sequence fields that may be null depending on the hardware type, as not all equipment requires network connectivity.

A required **status** character sequence tracks the current state of the hardware (e.g., available, occupied).

Like other entities, it includes a **created_at** timestamp for inventory tracking and administrative purposes.

0.5 Associations

This section details the relationships between entities, explaining how they interact within the system and what restrictions they have.

User associations include:

- **Token** - A weak association where each user may have multiple tokens (N:1). Tokens are automatically generated by the system as needed, with the user's primary key serving as part of the token's composite key. It is worth noting that an Administrator can also hold the position of Professor due to hierarchical considerations.
- **Group** - Users have two distinct relationships with groups:
 - **creates** - Users can create multiple groups, but each group has exactly one creator (N:1). Only Professors can create groups.
 - **belongs** - Users can belong to multiple groups, and groups can have multiple members (N:N)
- **Laboratory** - Users have three types of laboratory associations:
 - **creates** - Users can create multiple laboratories, but each laboratory has one creator (N:1). Only Professors can create laboratories.
 - **joins** - Users can join multiple laboratories over time, and laboratories can be joined by multiple users (N:N). While the application domain may impose restrictions (e.g., limiting concurrent laboratory access), the database structure maintains this flexibility to support historical tracking.
 - **session** - Users can have multiple sessions. The exact number of concurrent sessions is defined in the application domain. This association also includes the hardware entity, since they are related to each other. This association contains three important attributes: **start_time** and **end_time**, both as timestamps for session control and statistics, and a **state** attribute. The state attribute is a char type and can be *P* (In Progress), *C* (Completed), or *S* (Scheduled).

Group has an additional association with Laboratory, controlling laboratory access permissions. Groups can be associated with multiple laboratories, and laboratories can be accessible to multiple groups (N:N).

Laboratory maintains these additional associations:

- **Lab Session** - Laboratories can have multiple sessions, but each session is associated with exactly one laboratory (N:1)
- **Hardware** - Laboratories can be assigned multiple pieces of hardware, and hardware can be assigned to multiple laboratories over time (N:N)

Note that additional constraints and business rules are implemented at the web API level. Please refer to the web API documentation for detailed information about these restrictions.

Bibliography

- [1] PostgreSQL Global Development Group. *PostgreSQL Documentation*. [Online]. Available: <https://www.postgresql.org/docs/>